

TASK Géométrie

unités, coordonnées ISU, géométrie, grille, cellules et entités

Squelettes de classes

Les squelettes de classes sont fournis dans le répertoire `SKL/`.

- sans nom de `package`
- sans les `import`
- sans attributs : `final`, `static`, `public`, `abstract`, ...
- sans les *body* des méthodes
- Les variables sont suivies d'une unité `_unité` pour faciliter la mise au point.
 - `width_ncell` est la largeur du monde exprimée en *nombre de cellules*
 - `width_cm` est la largeur du monde exprimée en *cm*

Dans un premier temps, on considère deux *packages*

- `package engine` : contiendra ce qui concerne le moteur
- `package game` : contiendra ce qui est spécifique à un jeu.

Game

- `Game` contient les paramètres de configuration du moteur pour un jeu particulier
- Dans la phase 1 du projet on réalisera le jeu *Pac Man*
- Dans la phase 2 du projet cela sera votre jeu

To Do

Répondez aux questions *avant de* commencer à compléter le squelette de code.

- QUESTIONS
 - Dans quel *package* mettre `Game` ?
 - La classe `Game` est-elle *abstraite* ou *concrète* ?
 - Comment s'assurer que les dimensions en «cm» et en «nombre de cellules» sont cohérentes ?
 - Quelle contrainte (sous forme d'assertion) faut-il mettre ? Où placer les assertions ?

- Commencez à compléter `Game`
- Demandez-vous systématiquement quelle visibilité donner à chaque champs et chaque méthode : `private` , `protected` , `package visibility` ou `public` ?
- Écrivez des vérifications à l'aide d'assertions
 - `assert( width_ncell * cmPerCell == width_cm );`
- Testez.
- Validez avec un enseignant.

GAME

```
class Game {
    // CONSTANT
    boolean torusOnXaxis = true; // vrai si l'axe X est une boucle fermée
    boolean torusOnYaxis = true; // vrai si l'axe Y est une boucle fermée
    double cmPerCell = 3.7; // échelle qui relie l'unité ncell à cm
    int pixelPerCm = 2; // échelle qui relie l'unité pixel à cm
    // FIELDS
    int width_ncell; // largeur du monde en nombre de cellules
    int height_ncell; // hauteur du monde en nombre de cellules
    double width_cm; // largeur du monde en cm
    double height_cm; // hauteur du monde en cm
    Grid grid; // permet la création de coordonnées en unités ncell
    ISU isu; // permet la création de coordonnées en unités cm
    Picture pict; // permet la création de coordonnées en unités pixel
    // CONSTRUCTORS
    Game(int w_ncell, int h_ncell) {}
    Game(double w_cm, double h_cm) {}
    // GETTER
    Game game();
    Game game() { return null; }
    // SHOW
    void show(PrintStream ps) {}
}
```

Main

- contient la fonction `main` qui lance le jeu *Pac Man*.

To Do

- Répondez aux questions puis commencez à compléter le squelette de code.
- QUESTIONS
 - Dans quel `package` mettre `Main` ?
 - La classe `Main` est-elle *abstraite* ou *concrète* ?

- Commencez à compléter `Main` pour qu'elle crée un jeu, instance de la classe `Game`.

MAIN

```
class Main {  
    void main(String args[]) {}  
}
```

Axis définit la géométrie du monde du jeu

- De nombreux jeux, tels que *PacMan*, se déroule sur un Tore, *i.e.*, une feuille de papier dont on recolle les bords opposés
 - L'axe des y est replié : le bord du haut coïncide avec celui du bas
 - L'axe des x est replié : le bord droit coïncide avec le bord gauche.
- Les longueurs et la position sur un axe dépendent de la géométrie.

Sur un tore de périmètre 5, la position $x=7$ est identique à la position $x=2$: on a fait un tour.

- Les distances sont impactées par la géométrie du Tore

En effet, sur un Tore (ou une sphère) la distance la plus courte est calculée en choisissant la meilleure des directions possibles (cf. cours)

- La géométrie est capturée par les fonctions `normalize` et `distance`.

To Do

- QUESTIONS
 - Dans quel *package* mettre `Axis` ?
 - Pour créer un objet qui représente un Tore 2D, combien faut-il définir d'instances d' `Axis` ?
 - Doit-on créer des instances de `Axis` ou bien `Axis` est-elle une collection de fonctions ?
 - Quelle contrainte (sous forme d'assertion) faut-il imposer sur `perimeter` ?
 - où placer cette assertion ?
- Commencez à compléter `Axis`.
- Demandez-vous systématiquement quelle visibilité donner à chaque champs et chaque méthode : `private`, `protected`, *package visibility* ou `public` ?
- Écrivez des vérifications à l'aide d'assertions.

- `assert( i == normalize(i + k * perimeter) );`

- Écrivez des tests. Testez.
- Validez avec un enseignant.

AXIS

```
class Axis {
    // FIELDS
    boolean onTorus;
    double perimeter;
    double halfPerimeter;
    // CONSTRUCTOR
    Axis(boolean onTorus, double perimeter) {}
    // NORMALIZE INTEGER LENGTH
    int normalize(int length) { return 0; }
    int modp(int length, int perimeter) { return 0; }
    // NORMALIZE REAL LENGTH
    double normalize(double length) { return 0.0; }
    double modp(double length, double perimeter) { return 0.0; }
    // DISTANCE
    double distance(double position1, double position2) { return 0.0; }
}
```

Grid

- Une grille contient un tableau 2D de cellules.
- La classe `Grid` définit 4 *inner classes* : `Cell`, `Dimension`, `Vector`, `Position`

À quoi sert la grille ? à identifier ses voisins

- Pour connaître les voisins d'une entité on consulte
 - la cellule dans laquelle se trouve le centre de l'entité
 - les cellules voisines

Selon le jeu

- une cellule peut contenir 0, 1 ou plusieurs entités.
- Une entité peut occuper toute la cellule : `Entity.size == Cell.size`
- Plusieurs entités peuvent se partager une cellule sans se toucher si : `Entitty.size << Cell.size`
- Une entité `Boss` peut occuper plusieurs cellules : `Boss.size >> Cell.size`

Dans tous les cas, l'entité doit pouvoir indiquer :

- quelles sont les cellules qu'elle occupe,

- dans quelle cellule se trouve son centre de masse,
- quelles sont les cellules voisines, *i.e.* les cellules qui entourent les cellules occupées.

L'unité `_ncell` = «nombre de cellules»

- l'unité utilisée dans la grille `Grid` est le «nombre de cellules», notée explicitement `_ncell` dans le code.
- une `Grid.Dimension` possède une composante `x_ncell` et une composante `y_ncell`, en unités `Grid`
- une `Grid.Position` (x,y) est un point de la grille, en unités `Grid`.
- un `Grid.Vecteur` (x,y) est un vecteur, en unités `Grid`.
- une cellule de la grille, `Grid.Cell`, a une `Grid.Position`
 - toutes les cellules ont la même dimension, nécessairement 1×1 `_ncell`.

Différences entre *dimension*, *point* et *vecteur*

- Les dimensions sont impactées par la géométrie (cf. `normalize` décrite au paragraphe suivant).
- Les dimensions n'ont pas d'autres opérateurs que `normalize`. On peut juste les construire et lire leur composante x et leur composante y.
- Les vecteurs peuvent être additionnés (`add`).
- On ne peut pas additionner deux points. Par contre, un point peut être translaté par un vecteur (`translate`).

To Do

- QUESTIONS
 - Les inner classes *point*, *dimension*, et *vecteur* sont-elles `static` ou non ?
 - Quelles sont les relations d'extensions entre *point*, *dimension*, *vecteur* ?
 - Quelles méthodes devraient avoir l'annotation `@Override` ?
- Commencez à compléter `Grid`.
- Demandez-vous systématiquement quelle visibilité donner à chaque champ et chaque méthode : `private`, `protected`, *package visibility* ou `public` ?
- Écrivez des vérifications à l'aide d'assertions.
 - `assert( cellAt(p).position().equiv(p) );`
- Écrivez des tests. Testez.
- Validez avec un enseignant.

GRID

```

class Grid {
    // FIELDS
    ISU isu;
    Axis xAxis, yAxis;
    int width_ncell, height_ncell;
    Cell[][] grid;
    // CONSTRUCTOR
    Grid(Game game) {}
    // INIT
    void init() {}
    // GETTER
    int width() { return 0; }
    int height() { return 0; }
    Grid.Cell cellAt(Grid.Position p) { return null; }
    // SHOW
    void show(PrintStream ps) {}
}

```

DIMENSION (nb cell)

```

class Dimension {
    int x_ncell, y_ncell;
    // CONSTRUCTOR
    Dimension(int x_ncell, int y_ncell) {}
    // GETTER
    int x() { return 0; }
    int y() { return 0; }
    // GEOMETRY
    void normalize() {}
    // EQUALS / EQUIV
    boolean equals(Object o) { return false; }
    boolean equiv(Dimension d) { return false; }
    // CONVERSION
    ISU.Dimension toISUDimension() { return null; }
    // SHOW
    void show(PrintStream ps) {}
}

```

VECTOR

```

class Vector {
    // CONSTRUCTOR
    Vector(int x_ncell, int y_ncell) {}
    // OPERATION
    void add(Vector v) {}
    // SHOW
    void show(PrintStream ps) {}
}

```

POINT

```

class Position {
    // CONSTRUCTOR
    Position(int x_ncell, int y_ncell) {}
    // COPY ? if needed
    Grid.Position copy() { return null; }
    // EQUALS
    boolean equals(Object o) { return false; }
    // TRANSLATION
    void translate(Vector v) {}
    void moveNorth(int n_ncell) {}
    // ROTATION ? if needed
    void rotateAround(Grid.Position position, int angle_degree) {}
    // DISTANCE
    double distanceTo(Position p) { return 0.0; }
    // CONVERSION
    ISU.Coord toISUCoord() { return null; }
    ISU.Coord toISUCoordCentered() { return null; }
    Picture.Pixel toPicturePixel() { return null; }
    // SHOW
    void show(PrintStream ps) {}
}

```

CELL

```

class Cell {
    Grid.Dimension size;
    Grid.Position position;
    List<Entity> entities;
    // CONSTRUCTOR
    Cell(Position p) {}
    // ADD
    void add(Entity e) {}
    // REMOVE
    void remove(Entity e) {}
    // PREDICATE
    boolean contains(Entity e) { return false; }
    // SHOW
    void show(PrintStream ps) {}
}

```

Conversion entre « ncell » et « cm »

Nous aurons besoin d'une correspondance entre les coordonnées (en cm) dans la carte et les positions (en « nombre de cellules ») dans la grille.

- On considère des cellules carrées avec *hauteur = largeur = 1 ncell*
- Une cellule connaît sa position dans la grille
- À partir de la position de la cellule, on peut obtenir

- la coordonnée (en cm) du coin en haut à gauche de la cellule
- la coordonnée (en cm) du centre de la cellule
- le facteur d'échelle `Game.cmPerCell` est la dimension d'une cellule en cm.
- Le facteur d'échelle `Game.cmPerCell` permet de définir des conversions
 - entre `Grid.Position` et `ISU.Coord`
 - entre `Grid.Dimension` et `ISU.Dimension`

ISU = International System of Units

À quoi servent les coordonnées ISU ? à connaître la position précise du centre d'une entité

Selon le jeu,

- savoir dans quelle cellule se trouve l'entité, ne suffit pas à déterminer avec précision la distance entre deux entités.
- pour savoir si deux entités sont en collision, il faut connaître la taille de l'entité et les coordonnées précises de son centre.

Les inner classes de ISU

- l'unité `ISU` est le «cm», notée explicitement `_cm` dans le code.
- c'est l'unité du monde simulé
- une `ISU.Dimension` possède deux composantes en unités `ISU`
- une `ISU.Coord` (x,y) est un point en unités `ISU`
- une `ISU.Vector` (x,y) est un vecteur en unités `ISU`

Afin de coller avec les conventions du graphique et de la grille, on décide que

- L'axe des x est orienté vers la droite
- L'axe des y est orienté vers le bas
- L'origine ($x_{cm}=0, y_{cm}=0$) du repère ISU correspond au centre de la cellule de coordonnées ($x_{ncell}=0, y_{ncell}=0$).

To Do

- QUESTIONS
 - Comment garantir qu'un couple (x_{cm}, y_{cm}) sera toujours normalisé ?
 - Comment éviter qu'on puisse modifier `x_cm` en oubliant de le normaliser ?
- Commencez à compléter `ISU` et continuez à compléter `Geometry`.
- Demandez-vous systématiquement quelle visibilité donner à chaque champs et chaque méthode : `private`, `protected`, `package visibility` ou `public` ?
- Définissez les conversions dans `Grid` et `ISU`.

- Écrivez des vérifications `check` à l'aide d'assertions.
- Testez.
- Validez avec un enseignant.

ISU

```
class ISU {
    // FIELDS
    Axis xAxis, yAxis;
    ISU isu;
    Grid grid;
    // CONSTRUCTOR
    ISU(Game game) {}
    // SETTER
    void set(Grid grid) {}
}
```

DIMENSION (cm)

```
class Dimension {
    double x_cm, y_cm;
    // CONSTRUCTOR
    Dimension(double x_cm, double y_cm) {}
    // GEOMETRY
    void normalize() {}
    // SETTER
    void setxy(double x_cm, double y_cm) {}
    // GETTER
    ISU isu() { return null; }
    // EQUALS / EQUIV
    boolean equals(Object o) { return false; }
    boolean equiv(Dimension d) { return false; }
    // GETTER
    double x() { return 0.0; }
    double y() { return 0.0; }
    // FACTORY
    ISU.Vector mkScaledVector(double factor) { return null; }
    ISU.Vector mkScaledVector(double xFactor, double yFactor) { return null; }
    ISU.Vector mkVector() { return null; }
    // SHOW
    void show(PrintStream ps) {}
}
```

POINT

```

class Coord {
    // CONSTRUCTOR
    Coord(double x_cm, double y_cm) {}
    // SHOW
    void show(PrintStream ps) {}
    // EQUALS
    boolean equals(Object o) { return false; }
    // FACTORY
    ISU.Vector mkVectorToward(Coord target) { return null; }
    // CONVERSION
    Grid.Position toGridPosition() { return null; }
    // TRANSLATION
    void translate(ISU.Vector v) {}
    ISU.Coord mkTranslated(ISU.Vector v) { return null; }
    // COPY
    ISU.Coord mkCopy() { return null; }
    // ROTATION
    void rotation(int angle_degree) {}
    void rotateAround(Coord center, int angle_degree) {}
    // DISTANCE
    double distanceTo(Coord pt) { return 0.0; }
}

```

VECTOR

```

class Vector {
    // CONSTRUCTOR
    Vector(double targetX_cm, double targetY_cm) {}
    // OPERATOR
    void add(Vector v) {}
    void scale(double factor) {}
    void scale(double xFactor, double yFactor) {}
    double dot(ISU.Vector v) { return 0.0; }
    double norm() { return 0.0; }
    void unity() {}
    // TURN
    void turn(int angle_degree) {}
}

```

Entity

- Une entité du jeu possède (au moins)
 - une dimension ;
 - une orientation (en degré) par rapport à l'axe des x ;
 - une position dans la grille, de type `Grid.Position` ;
 - les coordonnées de son centre, de type `ISU.Coord` ;
 - position et coordonnées *doivent toujours rester cohérentes et synchronisées*.
 - les dimensions d'un pas de déplacement

- Une entité peut se déplacer
 - sur la grille avec `translate(Grid.Vector)`
 - sur la carte avec `translate(ISU.Vector)`

To Do

- Commencez à compléter `Entity`.
- Demandez-vous systématiquement quelle visibilité donner à chaque champs et chaque méthode : `private`, `protected`, `package visibility` ou `public` ?
- Écrivez des vérifications à l'aide d'assertions.
- Écrivez des tests qui effectue des déplacements automatiques d'entités
 - un tour complet de tore dans chacun des directions et s'assure qu'on est revenu à la même position.
- Écrivez des tests. Testez.
- Validez avec un enseignant.

ENTITY

```
class Entity {
    // FIELDS
    Grid grid;
    ISU isu;
    String name;
    // FIELDS
    ISU.Dimension size; // dimension de l'entité
    ISU.Dimension step; // dimension d'un pas de déplacement
    Grid.Position position; // position en ncell dans la grille
    ISU.Coord center; // coordonnées en cm du centre de l'entité
    // FIELDS
    int orientation_degree; // orientation par rapport à l'axe des x
    // CONSTRUCTOR
    Entity(String name) {}
    // SETTER
    void setPosition(Grid.Position position) {}
    void setCoord(ISU.Coord center) {}
    void setSize(Grid.Dimension dimension) {}
    void setSize(ISU.Dimension dimension) {}
    // GETTER
    ISU.Coord center() { return null; }
    Grid.Position position() { return null; }
    int orientation() { return 0; }
    // TRANSLATION
    void translate(Grid.Vector v) {}
    void translate(ISU.Vector v) {}
    // TURN
    void turn(int angle_degree) {}
    // SHOW
    void show(PrintStream ps) {}
}
```

MOVE

```
void moveNorth(int nStep) {}  
void moveSouth(int nStep) {}  
void moveEast(double length_cm) {}  
void moveWest(double length_cm) {}  
}
```